Esse Gemini é fora da curva né ?

Skills no Claude Code: O Guia Definitivo Para Quem Quer Parar de Subutilizar a Ferramenta Mais Poderosa da IA Agentica

Eric Luque
Co-founder Ecotrace | Ajudo CTOs e Heads de Produto a tomar decisões de IA defensáveis — em 6 semanas, com...

April 18, 2026

Existe um recurso no Claude Code que a maioria dos desenvolvedores ignora ou usa de forma superficial. Skills. E o que me incomoda é que a documentação oficial trata isso como se fosse mais uma feature qualquer, quando na prática é o mecanismo que separa quem usa Claude Code como um autocomplete glorificado de quem usa como um engenheiro autônomo dentro do time.

Recentemente li um artigo do Mario Tort, engenheiro da Anthropic, onde ele compartilha como a equipe interna usa centenas de skills em produção. O que vou fazer aqui não é traduzir o artigo dele. É destrinchar cada conceito, adicionar o que aprendi na prática, e entregar para você um guia que você pode usar amanhã de manhã.

O que são Skills, de verdade

A primeira coisa que precisa morrer é a ideia de que skill é um arquivo Markdown com instruções. Isso é como dizer que um carro é uma cadeira com rodas. Tecnicamente não está errado, mas você perdeu a parte que importa.

Uma skill é uma **pasta**. E essa pasta pode conter scripts, assets, dados, templates, documentos de referência, exemplos de código, configurações e qualquer outro recurso que o agente precise para executar uma tarefa com competência. O arquivo Markdown principal funciona como o ponto de entrada, ele diz ao Claude quando e como usar aquele conjunto de recursos. Mas o poder real está no que vem junto.

Pense assim: se o Claude Code é um estagiário extremamente inteligente mas sem contexto da empresa, a skill é o onboarding completo que transforma esse estagiário em alguém produtivo no primeiro dia. Não é uma lista de regras. É um kit de sobrevivência.

A estrutura típica de uma skill se parece com isso:

```
.claude/skills/minha-skill/
  skill.md              # Ponto de entrada com instruções
  config.json           # Configurações do usuário
  references/           # Documentação de apoio
    api-docs.md
    architecture.md
  assets/               # Templates e boilerplate
    template-component.tsx
    migration-base.sql
  scripts/              # Scripts auxiliares
    validate.sh
    seed-data.py
  examples/             # Exemplos de uso
    basic-usage.md
    advanced-patterns.md
```

Quando o Claude invoca uma skill, ele tem acesso a tudo isso. Ele pode ler a documentação, executar os scripts, usar os templates como base e consultar os exemplos antes de tomar qualquer decisão. É a diferença entre pedir para alguém "fazer uma API REST" e entregar para alguém o guia de estilo da empresa, o template de projeto, os scripts de validação e três exemplos de APIs que já estão em produção.

As 9 categorias que cobrem 90% dos casos

Depois de olhar dezenas de skills (tanto as que a Anthropic usa internamente quanto as que vi na comunidade) fica claro que existem padrões que se repetem. São 9 categorias, e entender cada uma muda completamente como você pensa sobre automação com IA.

1. Referência de Bibliotecas e APIs

O problema que resolve: o Claude conhece bibliotecas populares, mas erra nos detalhes. Ele vai sugerir um método que existia na v2 mas foi removido na v3. Vai usar a assinatura errada de uma função interna. Vai ignorar uma limitação conhecida que não está na documentação oficial.

Como funciona: você cria uma skill que contém a documentação atualizada da biblioteca, com foco nas partes que o Claude erra com frequência. Não é pra duplicar a documentação inteira , é pra preencher as lacunas.

Exemplos práticos:

1. **billing-lib:** documentação de uma biblioteca interna de cobrança, com os edge cases que só quem usa em produção conhece
2. **internal-platform-cli:** subcomandos, flags e exemplos de uma CLI proprietária
3. **frontend-design:** regras do design system, incluindo os clichês que devem ser evitados (tipo usar Inter com gradiente roxo em tudo)

O que incluir:

- Pasta examples/ com casos de uso reais
- Seção de gotchas (erros comuns que o Claude comete)
- Versão atual da biblioteca e breaking changes recentes
- Padrões de uso aprovados vs. padrões que parecem corretos mas causam problemas

2. Verificação de Produto

O problema que resolve: o Claude pode escrever código que compila, passa nos testes unitários e ainda assim está errado. Um formulário de cadastro pode renderizar perfeitamente nos testes mas ter um campo que não aparece no viewport mobile. Um fluxo de checkout pode funcionar com dados ideais mas quebrar com um cartão de teste específico.

Como funciona: skills de verificação usam ferramentas externas (Playwright, Cypress, tmux) para testar o comportamento real do código. O Claude executa o fluxo, grava a sessão, e faz asserções programáticas sobre o resultado.

Exemplos práticos:

1. **signup-flow-driver:** abre um navegador headless, preenche o formulário de cadastro, verifica redirecionamento e criação do usuário
2. **checkout-verifier:** testa o fluxo completo de compra com cartões de teste do Stripe, incluindo cenários de falha
3. **tmux-cli-driver:** para CLIs que exigem um TTY real, usa tmux para simular a interação

Por que isso importa: essa categoria é provavelmente a mais subestimada. A maioria das pessoas para no "funciona nos testes". Mas a diferença entre código que passa nos testes e código que funciona em produção é exatamente onde os bugs mais caros moram. Dedicar tempo para construir skills de verificação robustas se paga em semanas, não meses.

3. Recuperação e Análise de Dados

O problema que resolve: o Claude não tem acesso aos seus dashboards, ao seu data warehouse, às suas métricas. Mas muitas tarefas de

desenvolvimento exigem contexto sobre o estado atual do sistema. Quanto tráfego essa rota recebe? Qual a taxa de erro das últimas 24 horas? Quantos usuários usam essa feature?

Como funciona: a skill fornece bibliotecas auxiliares para consultar seus sistemas de dados, junto com credenciais e instruções de workflow. O Claude usa essas ferramentas para buscar dados antes de tomar decisões de implementação.

Exemplos práticos:

1. **funnel-query:** junta eventos de diferentes fontes para montar funis de conversão
2. **cohort-compare:** analisa retenção e conversão entre cohorts de usuários
3. **grafana:** mapeia dashboards existentes para que o Claude saiba onde buscar métricas específicas

O que incluir:

- Scripts de consulta com credenciais já configuradas
- IDs de dashboards e queries salvas
- Instruções de como interpretar os resultados no contexto do negócio

4. Processos de Negócio e Automação de Time

O problema que resolve: todo time tem tarefas repetitivas que consomem tempo mas não exigem criatividade. Postar a daily no Slack com o resumo do que cada pessoa fez. Criar tickets seguindo um template específico. Gerar o recap semanal com PRs, deploys e tickets fechados.

Como funciona: a skill automatiza o processo inteiro, incluindo a coleta de informações de múltiplas fontes e a formatação do resultado.

Exemplos práticos:

1. **standup-post:** agrega informações do task tracker, GitHub e Slack para gerar a daily automaticamente
2. **create-ticket:** garante que todo ticket siga o schema correto e executa as ações pós-criação (notificação, labels, assignment)
3. **weekly-recap:** compila PRs mergeados, tickets fechados e deploys feitos durante a semana

Detalhe importante: essas skills frequentemente usam **logs persistentes**. Um arquivo standups.log dentro da pasta da skill (ou melhor, em ${CLAUDE_PLUGIN_DATA}) funciona como uma memória que sobrevive entre sessões. O Claude pode consultar postagens anteriores para manter consistência e evitar repetição.

5. Templates de Código e Scaffolding

O problema que resolve: cada time tem seus padrões. Quando alguém cria um novo serviço, uma nova migration, um novo componente, existem dezenas de decisões que já foram tomadas e que não precisam ser

repensadas toda vez. Estrutura de pastas, imports padrão, configurações de teste, CI/CD inicial.

Como funciona: a skill combina templates pré-prontos com scripts de scaffolding. O Claude não gera tudo do zero, ele parte do template e adapta conforme o que foi pedido.

Exemplos práticos:

1. **new-workflow:** scaffolding completo de um novo serviço, incluindo docker-compose, CI e testes
2. **new-migration:** template de migration com rollback, validação e seed data
3. **create-app:** aplicação interna pré-configurada com auth, logging e monitoring

Por que templates e não geração pura: quando o Claude gera código do zero, ele toma decisões que podem ou não estar alinhadas com os padrões do time. Quando ele parte de um template, as decisões estruturais já foram tomadas por humanos. O Claude foca na customização, que é onde ele realmente agrega valor.

6. Qualidade de Código e Code Review

O problema que resolve: manter padrões de qualidade em um time é difícil. Reviews são inconsistentes, regras de estilo são esquecidas, e patterns aprovados pelo time nem sempre são seguidos.

Como funciona: skills de qualidade podem funcionar de duas formas. Como ferramentas que o desenvolvedor invoca manualmente (tipo um /review que analisa o diff atual) ou como hooks que rodam automaticamente em momentos específicos (pré-commit, pré-push, ou via GitHub Actions).

Exemplos práticos:

1. **adversarial-review:** usa sub-agentes para fazer múltiplas rodadas de crítica no código, forçando o Claude a defender suas escolhas
2. **code-style:** enforcement de estilo com scripts determinísticos (não depende do "julgamento" do Claude)
3. **testing-practices:** guia de como testar diferentes tipos de código no contexto específico do projeto

O que faz a diferença: a melhor skill de review que vi combina regras determinísticas (linting, formatação, type checking) com análise semântica do Claude. O linter pega o any escapando. O Claude pega o endpoint que retorna dados sensíveis sem autenticação.

7. CI/CD e Deploy

O problema que resolve: deploy não é só git push. É monitorar o pipeline, esperar os checks passarem, lidar com conflitos de merge, decidir se faz rollback ou avança, e comunicar o resultado pro time.

Como funciona: skills de deploy orquestram todo o processo, frequentemente referenciando outras skills (como as de dados, para

buscar métricas de saúde) durante a execução.

Exemplos práticos:

1. **babysit-pr:** monitora o PR após o push, retenta checks flaky, resolve conflitos de merge, e avisa quando está pronto
2. **deploy-service:** faz deploy com shift gradual de tráfego (canary), monitorando métricas e fazendo rollback automático se necessário
3. **cherry-pick-prod:** gerencia cherry-picks para produção usando worktrees isoladas, evitando contaminar a branch principal

Por que essa categoria é poderosa: porque deploy é o tipo de tarefa que exige atenção constante mas pouca criatividade. É o caso de uso perfeito para um agente: monitorar, reagir a condições específicas, e escalar para um humano só quando realmente precisa.

8. Runbooks

O problema que resolve: quando algo quebra em produção, o engenheiro de plantão precisa investigar rapidamente usando múltiplas ferramentas. Logs, métricas, traces, status de dependências. Cada investigação segue um padrão semelhante, mas a pressão do incidente faz com que etapas sejam puladas.

Como funciona: a skill mapeia sintomas para ferramentas de investigação e produz um relatório estruturado no final.

Exemplos práticos:

1. **service-debugging:** dado um sintoma ("latência alta na API de pagamentos"), executa a sequência de investigação: verifica métricas, correlaciona com deploys recentes, analisa logs, e produz um diagnóstico
2. **oncall-runner:** recebe um alerta e executa a investigação inicial automaticamente, entregando pro engenheiro um resumo do que já foi verificado
3. **log-correlator:** rastreia uma requisição específica através de múltiplos serviços usando trace IDs

9. Operações de Infraestrutura

O problema que resolve: manutenção de infraestrutura envolve operações que podem ser destrutivas. Limpar recursos órfãos, atualizar dependências, investigar custos. São tarefas rotineiras mas que exigem cuidado.

Como funciona: a skill inclui guards de segurança que impedem ações destrutivas sem confirmação explícita.

Exemplos práticos:

1. **resource-orphans:** identifica recursos cloud que não estão mais em uso, mas pede confirmação antes de deletar qualquer coisa
2. **dependency-management:** gerencia atualizações de dependências com processo de aprovação

3. **cost-investigation:** analisa a fatura cloud e identifica gastos anômalos ou oportunidades de otimização

Guard rails são obrigatórios aqui. Nenhuma skill de infraestrutura deveria executar rm -rf, DROP TABLE, force-push, ou kubectl delete sem confirmação humana. Isso se implementa com hooks no PreToolUse que interceptam comandos perigosos.

Como Construir Skills Que Funcionam de Verdade

Categorias são úteis para entender o espaço. Mas o que separa uma skill útil de uma skill que ninguém usa é a qualidade da execução. Aqui estão as práticas que fazem diferença.

Não escreva o óbvio

O Claude Code já sabe muito sobre o seu codebase. Ele lê os arquivos, entende a estrutura, infere padrões. Se a sua skill repete informação que o Claude consegue derivar sozinho, você está desperdiçando contexto, e contexto é o recurso mais escasso que existe num LLM.

Foque no que o Claude **não pode inferir**:

- Decisões de design que não estão documentadas no código
- Restrições políticas ou organizacionais ("nunca toque nesse serviço sem avisar o time X")
- Gotchas que só quem operou o sistema em produção conhece
- Preferências estéticas que não são capturadas por linters

O artigo do Mario menciona o exemplo da skill frontend-design que, entre outras coisas, instrui o Claude a evitar clichês estéticos como "fonte Inter com gradiente roxo". Isso é exatamente o tipo de informação que o Claude não consegue derivar do código, é preferência, é cultura, é o tipo de coisa que você só sabe porque alguém do time apontou.

A seção de Gotchas é o conteúdo mais valioso

O Mario é direto sobre isso: "o conteúdo mais valioso de qualquer skill é a seção de gotchas". Concordo completamente. E vou além: se a sua skill não tem uma seção de gotchas, ela provavelmente não está resolvendo um problema real.

Gotchas são os erros que o Claude comete repetidamente. São os padrões que parecem corretos mas causam bugs sutis. São as armadilhas que um desenvolvedor experiente do time já conhece mas que um novato (humano ou IA) vai cair.

A prática é simples: toda vez que o Claude errar usando uma skill, adicione o erro à seção de gotchas. As melhores skills da Anthropic começaram com "poucas linhas e um único gotcha" e cresceram organicamente à medida que novos edge cases apareciam.

Exemplo de seção de gotchas:

```
## Gotchas

### Não use findOne sem .lean() em queries de leitura

O Mongoose retorna documentos completos por padrão. Em
queries de leitura onde

você não precisa do documento Mongoose (e quase nunca
precisa), use .lean().

Sem isso, cada query aloca ~3x mais memória do que o
necessário.

### O campo status aceita null no banco mas não na API

A migration original permitiu null, mas o schema da API
valida como required.

Se você setar status como null direto no banco, a próxima
leitura via API vai

retornar 500. Sempre use o valor default "pending".

### Testes de integração precisam rodar com --runInBand

Os testes compartilham a instância do banco de teste. Se
rodarem em paralelo,

vão interferir uns nos outros. Sempre use a flag --
runInBand no Jest.
```

Use a estrutura de pastas como ferramenta de context engineering

Essa é provavelmente a insight mais importante e menos óbvia do artigo inteiro. O sistema de arquivos não é só organização, é uma ferramenta de engenharia de contexto.

Quando o Claude lê uma skill, ele pode navegar pela estrutura de pastas. Se você coloca um documento de referência em, o Claude pode consultar esse documento quando precisar de detalhes específicos, sem que todo o conteúdo precise estar no arquivo principal da skill.

Isso é **progressive disclosure aplicado a prompts de IA**. O arquivo principal da skill dá a visão geral e as regras. As subpastas contêm os detalhes que o Claude consulta sob demanda. Você controla o fluxo de informação sem estourar o contexto.

Estrutura recomendada:

```
references/     # Docs que o Claude consulta quando
precisa de detalhes

scripts/        # Código executável que o Claude pode
rodar
```

```
examples/       # Casos de uso concretos para o Claude se
basear

assets/         # Templates, configs, boilerplate
```

Mantenha flexibilidade, não seja um micromanager de IA

É tentador escrever skills super detalhadas com instruções passo a passo para cada cenário possível. Resista. Skills muito rígidas quebram assim que aparece um caso que você não previu, e casos imprevistos são a norma, não a exceção.

A regra é: dê ao Claude a informação necessária, mas deixe espaço para ele adaptar a abordagem ao contexto específico. Você quer um profissional informado, não um robô seguindo um checklist.

Ruim:

```
1. Sempre crie o arquivo no diretório src/components

2. Use o nome PascalCase

3. Importe React no topo

4. Crie a interface Props primeiro

5. Export default no final
```

Bom:

```
Componentes seguem a convenção do projeto: PascalCase, em
src/components/,

com interface de Props tipada. Veja examples/component-
pattern.tsx para o

padrão atual. Adapte conforme a complexidade do
componente.
```

A segunda versão dá a mesma informação mas permite que o Claude tome decisões inteligentes quando o caso não se encaixa perfeitamente no padrão.

Configure com config.json, não com hardcode

Muitas skills precisam de informações que variam entre usuários ou projetos. URLs de serviços, nomes de branches, credenciais, preferências

pessoais. Hardcodar essas informações torna a skill inflexível e difícil de compartilhar.

O padrão que funciona: armazene configurações em um config.json dentro do diretório da skill. Se o arquivo não existir, o Claude pergunta ao usuário usando a ferramenta AskUserQuestion.

```
{
  "slack_channel": "#eng-deploys",
  "default_branch": "main",
  "staging_url": "https://staging.meuapp.com",
  "monitoring_dashboard": "grafana.internal/d/api-health"
}
```

Isso transforma uma skill pessoal em uma skill portável que qualquer pessoa do time pode usar com suas próprias configurações.

Escreva descriptions para o modelo, não para humanos

O campo description de uma skill não é um resumo. É o conjunto de condições que determina quando o Claude deve considerar invocar aquela skill. Essa distinção é crucial.

Ruim (resumo para humanos):

```
description: "Skill para fazer deploy de serviços na AWS"
```

Bom (trigger conditions para o modelo):

```
description: "Use quando o usuário pedir para deployar,
publicar ou subir um
serviço. Também quando mencionar staging, produção,
canary, rollback, ou
quando um PR for aprovado e precisar ir para produção."
```

A description é o mecanismo de discovery. Se ela não descreve com precisão quando a skill deve ser ativada, o Claude vai ignorá-la nos momentos em que ela seria mais útil -- ou invocá-la quando não deveria.

Persista dados com inteligência

Skills podem funcionar como pequenos bancos de dados. Um log de standups postadas, um registro de deploys feitos, um histórico de reviews. Isso dá ao Claude memória entre sessões e permite que ele mantenha consistência ao longo do tempo.

Mas atenção: **dados dentro do diretório da skill podem ser deletados durante atualizações**. Se a skill é distribuída como plugin e recebe um update, tudo dentro da pasta pode ser sobrescrito.

A solução é usar ${CLAUDE_PLUGIN_DATA} - um diretório estável por plugin que sobrevive a atualizações. Dados que precisam persistir vão para lá. Templates e scripts que fazem parte da skill ficam no diretório da skill.

Inclua scripts reutilizáveis

Scripts auxiliares mudam a dinâmica de como o Claude trabalha. Sem scripts, o Claude precisa escrever cada operação do zero. Com scripts, ele se concentra em **composição** -- decidir o que fazer e em que ordem -- em vez de reimplementar boilerplate.

Pense em funções utilitárias que o Claude pode chamar:

1. **scripts/validate.sh**: valida o estado do sistema antes de uma operação
2. **scripts/seed-data.py**: popula o banco com dados de teste
3. **scripts/notify.sh**: envia notificação para o canal correto

O Claude vai "compor" esses scripts, encadeando-os conforme a necessidade do momento. Ele gasta os passos de raciocínio em decisões de alto nível, não em como formatar uma mensagem de Slack.

Hooks condicionais: potência quando precisa, silêncio quando não

Hooks são comandos que executam em resposta a eventos específicos (pré-uso de ferramenta, pós-uso, etc). O problema é que hooks que rodam o tempo todo são irritantes. Se toda vez que você roda um comando o Claude pede confirmação extra, a produtividade desaba.

A solução são hooks que só ativam quando a skill é invocada. Exemplos:

1. **/careful:** ativa um hook que bloqueia comandos destrutivos (`rm -rf`, DROP TABLE, force-push, kubectl delete) durante a sessão
2. **/freeze**: bloqueia edições fora de um diretório específico

Esses hooks são opcionais, ativados sob demanda, e ficam ativos apenas durante a sessão. É o equilíbrio entre segurança e produtividade.

Distribuição: do Repositório ao Marketplace

Existem dois caminhos para compartilhar skills.

Caminho 1: Commit no repositório

Coloque as skills em **.claude/skills/** no repositório do projeto. Todo mundo que clonar o repo tem acesso. Simples, sem dependências externas, versionado junto com o código.

Melhor para: times pequenos, skills específicas do projeto, iteração rápida.

Caminho 2: Plugin no marketplace

Empacote a skill como parte de um plugin Claude Code e publique no marketplace. Qualquer pessoa pode instalar.

Melhor para: skills genéricas que servem para múltiplos projetos, times grandes, ferramentas que a comunidade pode usar.

Na Anthropic, o processo é orgânico. Não existe um time centralizado que cuida do marketplace. Skills úteis surgem naturalmente, são compartilhadas informalmente (via GitHub ou Slack), e quando atingem um nível de adoção, são promovidas para o marketplace oficial via PR.

O cuidado aqui é com curadoria. É fácil criar skills ruins ou duplicadas. Algum mecanismo de revisão antes da publicação é importante para manter a qualidade.

Composição: Skills que Referenciam Skills

Uma skill pode referenciar outra pelo nome. A skill de deploy pode chamar a skill de métricas para verificar a saúde do sistema antes de continuar. A skill de runbook pode chamar a skill de logs para correlacionar eventos.

Isso é poderoso, mas ainda não é formalmente gerenciado. Não existe um package.json de skills com versões e resolução de dependências. Por enquanto, funciona na base da convenção: se a skill X depende da skill Y, ambas precisam estar instaladas e disponíveis.

É uma área que vai evoluir. Mas mesmo sem tooling formal, a composição já funciona na prática e multiplica o valor de cada skill individual.

Medindo Adoção

Como saber se as skills do seu time estão sendo usadas? Use hooks de PreToolUse para logar quando uma skill é invocada. Com isso você consegue ver:

- Quais skills são mais usadas
- Quais estão abandonadas (e podem ser removidas ou melhoradas)
- Quais momentos do dia/semana as skills são mais acionadas
- Quais skills são invocadas mas não completam o fluxo (indicativo de problemas)

Dados de adoção alimentam decisões sobre onde investir na criação de novas skills e na melhoria das existentes.

O Estado Atual e Para Onde Isso Vai

Skills são um primitivo agente que ainda está em estágio inicial. O ecossistema está se formando, as melhores práticas estão sendo descobertas na prática, e a comunidade inteira está aprendendo junto.

O que eu vejo de tendência:

1. **Skills vão se tornar o principal mecanismo de customização de agentes de código.** Mais do que system prompts, mais do que fine-tuning, mais do que RAG. Porque skills combinam instruções, dados e ferramentas em um pacote coeso e versionável.
2. **Gerenciamento de dependências entre skills vai precisar de tooling.** Hoje é informal. Mas quando times tiverem 50+ skills com interdependências, vai precisar de algo mais robusto.
3. **Skills de verificação de produto vão se tornar padrão.** Hoje são raras. Mas o gap entre "código que compila" e "produto que funciona" é exatamente onde a IA mais erra. Skills que fecham esse gap vão ser obrigatórias.
4. **O marketplace vai explodir em volume e vai precisar de curadoria séria.** O mesmo ciclo que aconteceu com npm, VS Code extensions e GitHub Actions vai acontecer com skills. Muita coisa ruim, pouca coisa excelente, e a descoberta vai ser o gargalo.

Por Onde Começar

Se você nunca criou uma skill, comece pequeno:

1. **Identifique uma tarefa que você explica pro Claude repetidamente.** Toda vez que você precisa dar contexto adicional antes do Claude acertar, existe uma skill ali.
2. **Crie uma skill mínima com um único gotcha.** Literalmente um arquivo Markdown com a instrução e um erro que o Claude comete naquele contexto.
3. **Use a skill por uma semana e adicione gotchas conforme eles aparecem.** As melhores skills crescem organicamente.
4. **Evolua para pastas com scripts e referências quando a complexidade justificar.** Não over-engineer no começo.
5. **Compartilhe com o time quando a skill estiver estável.** Commit no repositório. Veja se outros adotam.

A beleza das skills é que elas recompensam investimento incremental. Cada gotcha adicionado, cada script incluído, cada exemplo anexado torna a skill marginalmente melhor. E esses ganhos marginais se acumulam. Em algumas semanas, você tem uma skill que transforma a capacidade do Claude em uma área específica de forma que nenhum prompt avulso conseguiria.

O Efeito Opus 4.7: Por Que Skills Acabam de Se Tornar 10x Mais Importantes

Dois dias atrás, em 16 de abril de 2026, a Anthropic lançou o Claude Opus 4.7. E quando você cruza as capacidades desse novo modelo com tudo

que discutimos sobre skills, a conclusão é inevitável: o que era uma boa prática acaba de virar obrigação.

Vou explicar por quê.

O modelo que pensa antes de agir

A mudança comportamental mais significativa do Opus 4.7 em relação ao 4.6 se resume em uma frase que aparece nos benchmarks: **"4.7 pensa mais e age menos."** O número de tool calls por tarefa caiu. A taxa de erros em ferramentas caiu para um terço do que era. O modelo não fica mais preso em loops infinitos tentando a mesma abordagem que não funciona.

O que isso significa para skills? Significa que o Claude agora é **muito melhor em ler, interpretar e seguir** as instruções de uma skill antes de sair executando. No 4.6, era comum o modelo ler a skill superficialmente e partir para a ação. No 4.7, ele absorve o contexto, planeja, e só então age. Uma skill bem escrita agora tem um retorno muito maior porque o modelo realmente a utiliza como base de decisão, não como formalidade.

Auto-verificação nativa: skills de verificação ficam mais poderosas

Uma das capacidades mais impressionantes do Opus 4.7 é a **auto-verificação proativa**. O modelo agora escreve testes, roda, corrige falhas e só então reporta o resultado, tudo sem que você peça. Nos benchmarks, ele chegou a rodar o output de um motor de text-to-speech que construiu autonomamente através de um speech recognizer para verificar se o resultado estava correto.

Lembra da categoria 2: Skills de Verificação de Produto? Elas acabaram de ganhar um motor muito mais potente. Quando você combina uma skill que define *o que verificar* (fluxo de signup, checkout, cenários de erro) com um modelo que já tem o instinto de *como verificar* (rodar, testar, validar antes de declarar pronto), o resultado é código que chega mais perto de "funciona em produção" do que qualquer versão anterior conseguia.

A melhoria na resolução de imagem, de 1.15 megapixels para 3.75 megapixels, também impacta diretamente aqui. Skills que usam Playwright para capturar screenshots de UI agora podem ser analisadas pelo Claude com 3x mais detalhe visual. Um botão mal posicionado, um texto truncado, um ícone fora de alinhamento, coisas que o 4.6 não enxergava, o 4.7 vê.

Memória de filesystem: o mecanismo que skills já usavam, agora turbinado

O artigo do Mario fala sobre usar logs persistentes e ${CLAUDE_PLUGIN_DATA} para dar memória às skills. O Opus 4.7 leva isso a outro nível porque o modelo agora é **nativamente melhor em usar memória baseada em filesystem**.

A Anthropic descreve isso explicitamente: "Opus 4.7 is better at writing and using file-system-based memory. If an agent maintains a scratchpad, notes file, or structured memory store across turns, that agent should improve at jotting down notes to itself and leveraging its notes in future tasks."

Traduzindo: skills que mantêm estado, log de standups, histórico de deploys, registro de decisões de code review, vão funcionar dramaticamente melhor. O Claude não só lê esses arquivos como agora

ativamente os atualiza e os consulta em tarefas futuras sem precisar ser instruído a fazê-lo.

Isso muda a categoria 4 (Processos de Negócio) e a categoria 7 (CI/CD) de forma fundamental. Uma skill de deploy que mantém um log de deploys anteriores agora alimenta um modelo que realmente usa esse histórico para tomar decisões melhores, "o último deploy desse serviço causou aumento de latência, vou monitorar essa métrica com mais atenção dessa vez."

Interpretação literal: descriptions e gotchas importam mais do que nunca

O Opus 4.7 interpreta instruções de forma mais literal. Onde o 4.6 "preenchia lacunas" implícitas, o 4.7 segue exatamente o que está escrito. **Isso é uma faca de dois gumes e o meu artigo anterior fala sobre isso. Leia aqui.**

O lado bom: se a sua skill tem instruções claras e bem escritas, o modelo vai segui-las com precisão cirúrgica. Gotchas vão ser respeitados. Restrições vão ser obedecidas. Padrões vão ser seguidos.

O lado perigoso: se a sua skill tem instruções ambíguas, o modelo não vai mais "adivinhar" o que você quis dizer. Ele vai interpretar literalmente e o resultado pode ser inesperado.

A implicação prática é direta: **revise suas skills**. Cada instrução que dependia do Claude "entender o espírito" precisa ser reescrita para comunicar exatamente o que você quer. A seção de gotchas, que já era o conteúdo mais valioso, agora precisa ser ainda mais explícita.

Aquela prática que discutimos de escrever descriptions como trigger conditions e não como resumos? No 4.7, isso deixou de ser "boa prática" e virou "requisito". O modelo interpreta a description ao pé da letra para decidir se invoca a skill. Uma description vaga vai resultar em skills que nunca são invocadas nos momentos certos.

Task budgets: controle fino sobre quanto o agente gasta em cada skill

O Opus 4.7 introduz **task budgets** em beta, a capacidade de definir um orçamento de tokens para um loop agente completo. Isso inclui pensamento, tool calls, resultados e output final.

Para quem constrói skills, isso é game-changer. Você pode agora calibrar quanto "esforço computacional" uma skill deve consumir. Uma skill de scaffolding simples pode rodar com budget baixo. Uma skill de debugging complexa pode receber budget alto. Uma skill de runbook para incidentes de produção pode usar o máximo disponível.

Combinado com o novo nível de esforço xhigh (entre high e max), você tem controle granular sobre o trade-off entre profundidade de raciocínio e velocidade. Skills críticas rodam em xhigh. Skills de rotina rodam em high. Você decide.

Coordenação multi-agente: composição de skills ganha um motor novo

Uma limitação que mencionamos é que a composição de skills (uma skill chamando outra) ainda não tem tooling formal. Mas o Opus 4.7 traz melhorias em coordenação multi-agente que tornam essa composição mais robusta na prática.

O modelo agora é melhor em orquestrar workstreams paralelos, delegar sub-tarefas, e manter coerência entre agentes. Uma skill de deploy que dispara uma skill de métricas em paralelo com uma skill de notificação vai funcionar com menos atrito do que no 4.6.

O detalhe importante: o 4.7 é mais conservador em disparar sub-agentes por padrão. Ele só faz quando explicitamente instruído. Isso significa que suas skills precisam ser claras sobre quando e como a composição deve acontecer. Mais um motivo para descriptions e instruções precisas.

Os números que importam

Para contextualizar o impacto prático:

Métrica	Opus 4.6	Opus 4.7	Delta
CursorBench	58%	70%	+12pp
Rakuten-SWE-Bench (produção)	baseline	3x resolução	+200%
Erros de tool calls (Notion Agent)	baseline	1/3 dos erros	-66%
Visual acuity (XBOW)	54.5%	98.5%	+44pp
Factory Droids (tarefas complexas)	baseline	+10-15%	+10-15%

Esses números são sobre o modelo sem skills customizadas. Agora imagine o multiplicador quando você adiciona skills bem construídas em cima de um modelo que já é 3x melhor em resolver tarefas de produção.

O ponto cego: breaking changes que afetam skills existentes

Nem tudo são flores. O Opus 4.7 traz mudanças que podem quebrar skills existentes:

1. **Parâmetros de sampling removidos:** temperature, top_p, top_k retornam erro 400. Se alguma skill ou script auxiliar configurava esses parâmetros via API, vai quebrar.
2. **Tokenizer novo:** inputs de texto consomem de 1.0x a 1.35x mais tokens. Skills com documentos de referência pesados vão custar mais. Imagens em alta resolução saltam de ~1.600 para ~4.784 tokens. Isso afeta diretamente o budget de contexto.
3. **Prefill bloqueado:** assistant message prefilling retorna erro 400. Scripts que dependiam dessa técnica precisam ser reescritos.

A recomendação: passe por cada skill do seu repositório e verifique se alguma depende desses comportamentos. O custo de descobrir isso em produção é alto.

A tese final

O Opus 4.7 não apenas melhora skills existentes, ele muda o cálculo de ROI de criar skills novas. Quando o modelo era bom mas impreciso, uma skill mediocre ainda ajudava. Agora que o modelo é excelente em seguir instruções, a qualidade da skill é o fator limitante.

A analogia mais precisa: no 4.6, uma skill era como dar um mapa para alguém que enxergava razoavelmente bem. No 4.7, é como dar um mapa

para alguém com visão perfeita. Se o mapa estiver errado, essa pessoa vai seguir as instruções erradas com muito mais convicção.

Invista tempo em skills bem escritas, com gotchas atualizados, descriptions precisas, e estruturas de pasta que aproveitem o progressive disclosure. O modelo agora recompensa esse investimento de forma desproporcional.

> Este artigo foi inspirado pelo guia original de Mario Tort, engenheiro da Anthropic. As ideias centrais sobre skills são dele. A interpretação, os exemplos adicionais, a análise do Opus 4.7 e as opiniões sobre o futuro são minhas.

Comments

44

Like Comment Share

Add a comment...

No comments, yet.

Be the first to comment.

Start the conversation

Eric Luque

Co-founder Ecotrace | Ajudo CTOs e Heads de Produto a tomar decisões de IA defensáveis — em 6 semanas, com critério que o board aceita.

Followed by Eder, Luisa and 5 others you know

✓ Following

More articles for you

Vibe Coding — 5 coisas que aprendi depois de incorporar mais de 30 mil linhas de código geradas por IA no meu projeto de...

Tomás Hugo Almeida

AITDD - Uma abordagem para desenvolvimento com AI

Fabio Omachi

49 · 2 comments · 1 repost

Desvendando o Poder Oculto: Low-Code, No-Code e IA – A Tríade que Acelera a Inovação em Análise de Dados!

Rafael Dias

35 · 2 comments

4